BittiBiomi

Paperinen työpaketti

Oma Minecraft-teemapaketti: tekstuurit, mallit, äänet ja skriptit

Viikot 34–49 · syysloma vko 42 · luovutus pe 4.12.2026

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Paketti julkaistaan avoimella lisenssillä ja repository on julkinen ensimmäisestä commitista. Älä laita julkiseen repositoryyn henkilötietoja, kotiosoitetta, koulun tunnisteita tai muiden nimiä — Git-historia on pysyvä. Sovi tekijänimi ohjaajan kanssa, ja alaikäisenä sovi julkisesta repositorystä myös huoltajan kanssa.

Mitä rakennat ja miten etenet

Valmis teemapaketti on julkinen GitHub-release, jonka kuka tahansa pelaaja lataa ja asentaa ohjeesi avulla. Asennuksen jälkeen peli näyttää ja kuulostaa teemaltasi, ja oma resepti avaa saavutuksen ja palkinnon.

1. Pelaaja löytää paketin julkisesta GitHub-releasesta ja lataa kaksi zip-pakettia. README kertoo peliversion, lisenssin ja asennuksen.
2. Hän asentaa resurssipaketin pelin resourcepacks-kansioon ja datapaketin maailman datapacks-kansioon pelkän ohjeesi avulla.
3. Pelissä näkyvät omat tekstuurit, 3D-mallit ja suomenkieliset nimet, ja paketin oma ääni kuuluu.
4. Kun pelaaja valmistaa teemasi reseptin, saavutus laukeaa ja palkintofunktio antaa palkinnon. LICENSE ja CREDITS kertovat, mitä toinen tekijä saa paketilla tehdä.

Projektin 5 vaihetta



1. Valmistelu ja rajaus · viikot 34–35. Selvität toimeksiannon, teet tyhjän paketin, joka näkyy pelissä, ja perustat julkisen repositoryn. Asset-pack-suunnitelma, lisenssi ja GitHub-issueiksi pilkottu sisältö rajaavat paketin ennen ensimmäistä pikseliä.

2. Paketin ilme ja ääni · viikot 36–40. Piirrät blokki- ja esinetekstuurit, annat sisällölle omat nimet, mallinnat Blockbenchillä ja lisäät äänen. Jokainen asset kulkee saman asset-työkierron läpi, ja resurssipaketti on koossa ennen katselmointia.

3. Palaute ja pelillinen lisä · viikot 41–44. Testaajat kokeilevat väliversiota, ja teet heidän palautteestaan sovitun muutoksen omassa haarassa. Datapaketti tuo reseptin, saavutuksen ja palkintofunktion, joten paketin koko pakollinen sisältö on valmis.

4. Testaus ja laatu · viikot 45–46. Testaat koko paketin kirjatuilla testitapauksilla, korjaat kolme virhettä ketjuna ja siistit rakenteen. Lisenssi ja lähteet tarkistetaan, jotta paketin saa julkaista avoimena.

5. Julkaisu ja näyttö · viikot 47–49. Kaksi testaajaa asentaa julkaisuehdokkaan pelkän ohjeen avulla, ja v1.0 julkaistaan GitHub-releasena. Lopuksi kokoat näyttöaineiston ja esittelet työsi demossa.

Viikon työvaihe on tämän sivuston työohje. Yhden assetin tai muutoksen teet Työtapa-sivun asset-työkierrolla, ja sen arvio ja Valmis kun -ehto ovat GitHub-issuessa. Testien tulokset ja päätökset kirjaat projektipäiväkirjaan, ja valmiin issuen kommenttiin lisäät commit-linkin.

Aikataulu

Vko	Pvm	Viikon aihe	Vaihe
34	17.–21.8.	Aloitus: teema ja työkalut	1
35	24.–28.8.	Asset-pack-suunnitelma: teema ja sisältölista	1
36	31.8.–4.9.	Asset 1 · Ensimmäiset blokkitekstuurit peliin	2
37	7.–11.9.	Asset 2 · Esinetekstuurit ja omat nimet	2
38	14.–18.9.	Asset 3 · Ensimmäinen Blockbench-malli	2
39	21.–25.9.	Asset 4 · Isompi malli tai hahmon uusi ilme	2
40	28.9.–2.10.	Asset 5 · Äänet ja tunnelma	2
41	5.–9.10.	Katselmointi · Ensimmäiset testaajat	3
42	12.–16.10.	Syysloma – ei projektityötä	–
43	19.–23.10.	Asset 6 · Palautemuutos ja datapaketin runko	3
44	26.–30.10.	Asset 7 · Skriptattu ominaisuus	3
45	2.–6.11.	Paketin laatu · Paketti kestää käyttöä	4
46	9.–13.11.	Paketin laatu · Rakenne, jonka toinen ymmärtää	4
47	16.–20.11.	Julkaisu · Julkaisuehdokas RC1 testissä	5
48	23.–27.11.	Julkaisu · v1.0 julki	5
49	30.11.–4.12.	Näyttö · Aineisto ja luovutus	5

Palautus viimeistään pe 4.12.2026. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe 1 – Valmistelu ja rajaus

Vko 34 · 17.–21.8. – Aloitus: teema ja työkalut

Yhteys kokonaisprojektiin: Tämä on projektin ensimmäinen viikko, joten pohjana on vain toimeksianto. Selvität ohjaajalta, mille peliversiolle ja kenelle paketti tehdään, ja teet tyhjän paketin, joka näkyy pelissä. Kun kansiorakenne ja pack_format toimivat nyt, jokainen tuleva asset voidaan viedä samaan pakettiin ja testata heti.

Viikon tavoite: Oma tyhjä paketti näkyy pelin Resource Packs -valikossa omalla kuvallaan, ja julkinen repository on perustettu.

GitHub Desktop: kuvaohje ilman Git-komentoja

Työvaihe 1 / 5 · Lue toimeksianto ja kirjoita kuusi kysymystä ohjaajalle.

- ☐ Avaa sivun Toimeksianto-näkymä ja lue teksti kerran alusta loppuun.
- ☐ Kirjoita muistiin jokainen asia, joka paketissa on pakko olla, esimerkiksi omat tekstuurit, 3D-malli ja omat nimet.
- ☐ Kirjoita muistiin asiat, joita teksti ei kerro. Esimerkiksi: mille Minecraft-versiolle paketti tehdään?
- ☐ Kirjoita kuusi kysymystä ohjaajalle. Jokaisen vastauksen pitää auttaa sinua päättämään jotain paketista.
- ☐ Kirjoita jokaisen kysymyksen perään, mitä päätät vastauksen perusteella.

Valmis kun: Sinulla on kuusi erilaista kysymystä, ja jokaisen perässä lukee, mitä päätät vastauksen perusteella.

Tallenna työnäyte: Kysymyslista viikon 34 päiväkirjaan, kenttään Mitä tein ja miten?

Työvaihe 2 / 5 · Pidä aloituskeskustelu ja kirjaa ohjaajan vastaukset.

- ☐ Sovi ohjaajan kanssa aloituskeskustelun aika.
- ☐ Kysy kysymykset yksi kerrallaan. Kirjoita vastaus heti ylös ohjaajan omin sanoin.
- ☐ Jos ohjaaja ei vielä tiedä vastausta, kirjoita kohtaan sana ”avoin”. Älä keksi vastausta itse tai tekoälyllä.
- ☐ Sovi, mille Minecraft Java -versiolle paketti tehdään. Kirjaa versio, esimerkiksi 1.21.8.
- ☐ Kirjaa päiväkirjaan keskustelun päivä ja osallistujien roolit. Älä kirjoita muiden ihmisten nimiä.

Valmis kun: Päiväkirjassa on jokaiselle kuudelle kysymykselle joko ohjaajan vastaus tai merkintä ”avoin”, ja Minecraft-versio on kirjattu.

Tallenna työnäyte: Vastaukset ja avoimet asiat viikon 34 päiväkirjaan.

Työvaihe 3 / 5 · Tutki yksi julkaistu teemapaketti ja kuvaa oma kohdeyleisösi.

- ☐ Avaa Modrinth tai Planet Minecraft ja etsi yksi julkaistu teemapaketti eli resurssipaketti, joka muuttaa pelin ilmettä yhden teeman mukaan.
- ☐ Kirjoita paketista kolme asiaa: kenelle se on tehty, mitä se sisältää ja millä lisenssillä se on julkaistu.
- ☐ Kirjoita yksi asia, jonka sinun pakettisi tekee toisin.
- ☐ Kirjoita yhdellä virkkeellä, kenelle oma pakettisi on tehty. Tämä on paketin kohdeyleisö.

Valmis kun: Päiväkirjassa on linkki tutkittuun pakettiin, kolme havaintoa, yksi ero ja oman paketin kohdeyleisö.

Tallenna työnäyte: Linkki ja havainnot viikon 34 päiväkirjaan.

Työvaihe 4 / 5 · Luo tyhjä resurssipaketti, joka näkyy pelin valikossa.

- ☐ Asenna Blockbench, VS Code ja GitHub Desktop (kuvaohjeen kohta 1). Tarkista Minecraft Launcherista, että sovittu Minecraft Java -versio käynnistyy.
- ☐ Luo koneellesi kansio resourcepack. Tee sinne VS Codella tiedosto pack.mcmeta avun mallin mukaan.
- ☐ Tarkista Minecraft Wikin Pack format -sivulta, mikä pack_format-arvo kuuluu sovitulle versiolle. Kirjoita se pack.mcmeta-tiedostoon.
- ☐ Piirrä Piskelillä 64 × 64 pikselin kuva paketin kuvakkeeksi ja tallenna se resourcepack-kansioon nimellä pack.png.
- ☐ Avaa pelissä Options → Resource Packs → Open Pack Folder. Kopioi resourcepack-kansio sinne.
- ☐ Valitse paketti valikossa ja siirrä se nuolella Selected-puolelle. Ota kuvakaappaus valikosta.

Valmis kun: Paketti näkyy pelin Resource Packs -valikossa omalla kuvalla ja kuvauksella, eikä peli varoita väärästä versiosta.

Tallenna työnäyte: Kuvakaappaus valikosta. Viet sen työvaiheessa 5 polkuun project-docs/evidence/week-34/valikko.png.

Työvaihe 5 / 5 · Perusta julkinen Git-repository turvallisesti.

- ☐ Lue avun lista siitä, mitä julkiseen repositoryyn ei laiteta. Git-historia on pysyvä. Piilota sähköpostisi GitHubissa: Settings → Emails → Keep my email addresses private.
- ☐ Sovi ohjaajan kanssa, millä tekijänimellä julkaiset. Jos olet alaikäinen, sovi julkisesta repositorysta myös huoltajan kanssa.
- ☐ Luo GitHubiin uusi repository: valitse näkyvyydeksi Public ja rastita Add a README file.
- ☐ Kirjaudu GitHub Desktopiin ja aseta tekijätietoihin sovittu tekijänimi ja noreply-osoite (kuvaohjeen kohta 2). Kloonaa repository: File → Clone repository → URL, eli kuvaohjeen reitti C.
- ☐ Siirrä resourcepack-kansio repositoryn kansioon. Luo sinne myös kansiot datapack ja project-docs sekä kuvaohjeen mallin mukainen .gitignore.
- ☐ Lisää datapack-kansioon tiedosto README.md, jossa lukee ”Datapaketti tehdään viikolla 43.” Tiedosto tarvitaan, koska Git ei tallenna tyhjää kansiota.
- ☐ Kirjoita repositoryn juuren README.md-tiedostoon paketin nimi ja yksi virke siitä, mitä se tekee. Lisää valikon kuva polkuun project-docs/evidence/week-34/valikko.png.
- ☐ Tee commit ja push: kirjoita GitHub Desktopin Summary-kenttään ”Pakettirunko”, paina Commit to main ja sitten Push origin.

Valmis kun: Julkinen repository näkyy GitHubissa, ja siinä ovat resourcepack, datapack, project-docs ja README.md. Henkilötietoja ei ole mukana.

Tallenna työnäyte: Repositoryn linkki ja ensimmäisen commitin tunniste (7 merkin koodi GitHubin commit-listassa) viikon 34 päiväkirjaan.

Viikon lopputarkistus: Ohjaajan vastaukset ja avoimet asiat ovat päiväkirjassa, tyhjä paketti näkyy pelin pakettivalikossa ja ensimmäinen commit näkyy GitHubissa.

Työnäyte Git-repositoryyn ennen rastia: kysymykset ja ohjaajan vastaukset päiväkirjassa, tutkitun paketin linkki, kuva paketista pelin valikossa, julkisen repositoryn linkki ja ensimmäinen commit.

Vko 35 · 24.–28.8. – Asset-pack-suunnitelma: teema ja sisältölista

Yhteys kokonaisprojektiin: Viikon 34 vastaukset ohjaajalta ovat suunnitelman pohja. Nyt päätät teeman, paletin ja lisenssin ja pilkot pakollisen sisällön GitHub-issueiksi. Hyväksytty rajaus kertoo jokaisena asset-viikkona, mitä teet seuraavaksi ja mistä voit luopua, jos aika loppuu.

Viikon tavoite: Ohjaaja hyväksyy paketin rajauksen: jokainen pakollinen asset on GitHubissa omana issuenaan.

Täytä Asset-pack-suunnitelma tällä sivulla

Avaa koko toimeksianto

Työvaihe 1 / 4 · Täytä asset-pack-suunnitelma ja vie se repositoryyn.

- ☐ Avaa Suunnitelma-näkymä ja lue esitäytetty osa: se tulee toimeksiannosta.
- ☐ Kirjoita paketin työnimi. Kuvaa kahdella tai kolmella virkkeellä teema, tunnelma, kohdeyleisö ja oma roolisi.
- ☐ Valitse tekstuuriresoluutio. Pelin oma koko on 16 × 16 pikseliä, ja sillä on helpointa aloittaa.
- ☐ Kirjoita väripaletti ja työkalut, esimerkiksi Lospecin paletti, Piskel ja Blockbench.
- ☐ Kirjoita pakollisen sisällön (P0) määrät: vähintään 5 tekstuuria (3 blokkia ja 2 esinettä) ja 1 Blockbench-malli. Muu pakollinen (P0) sisältö tulee suunnitelmaan valmiina. Perustele valintasi yhdellä tai kahdella virkkeellä.
- ☐ Paina painiketta Lataa asset-pack-suunnitelma.md. Siirrä tiedosto project-docs-kansioon, tee commit ja push.

Valmis kun: project-docs/asset-pack-suunnitelma.md näkyy GitHubissa, eikä yhdessäkään OMA-merkityssä kentässä ole tyhjää kohtaa.

Tallenna työnäyte: project-docs/asset-pack-suunnitelma.md ja commit-linkki viikon 35 päiväkirjaan.

Työvaihe 2 / 4 · Sovi lisenssi ohjaajan kanssa ja lisää LICENSE-tiedosto.

- □ Lue Toimeksianto-näkymästä lisenssivaihtoehdot: CC BY, CC BY-SA, CC0 ja MIT.
- □ Sovi ohjaajan kanssa, millä lisenssillä paketti julkaistaan. Kirjaa valinta Suunnitelma-näkymän OHJAAJA-kenttään.
- □ Luo GitHubissa repositoryn juureen uusi tiedosto: Add file → Create new file. Anna nimeksi LICENSE. Jos lisenssi on MIT, valitse Choose a license template.
- □ Jos lisenssi on CC-lisenssi, kopioi sen teksti Creative Commonsin sivulta LICENSE-tiedostoon. Paina lopuksi Commit changes ja GitHub Desktopissa Fetch origin ja Pull origin, jotta LICENSE tulee myös koneellesi.

Valmis kun: LICENSE-tiedosto on repositoryn juuressa ja vastaa ohjaajan kanssa sovittua lisenssiä.

Tallenna työnäyte: Linkki LICENSE-tiedostoon ja sovittu lisenssi viikon 35 päiväkirjaan.

Työvaihe 3 / 4 · Pilko pakollinen sisältö GitHub-issueiksi.

- □ Luo GitHubissa kolme labelia eli merkintää: Issues → Labels → New label. Anna nimiksi P0 pakollinen, P1 tärkeä ja P2 lisä.
- □ Tee jokaisesta pakollisesta assetista oma issue: Issues → New issue. Aloita otsikko verbillä, esimerkiksi Piirrä kolme blokkitekstuuria.
- □ Kirjoita jokaiseen issueen arvio (puoli päivää tai päivä) ja Valmis kun -ehto: mitä pelissä näkyy, kun tehtävä on tehty.
- □ Anna pakollisille issueille label P0 pakollinen. Pakollinen (P0) on sisältö, jota ilman pakettia ei voi julkaista.
- □ Kirjoita yksi tai kaksi lisäideaa omiksi issueikseen. Anna niille label P1 tärkeä tai P2 lisä. Tärkeä (P1) tehdään, kun pakolliset toimivat. Lisä (P2) voi jäädä pois.
- □ Näytä lista ohjaajalle ja pyydä hyväksyntä rajaukselle. Kirjaa hyväksyjän rooli ja päivä päiväkirjaan.

Valmis kun: Jokaisella issuella on arvio, Valmis kun -ehto ja yksi label: P0 pakollinen, P1 tärkeä tai P2 lisä. Ohjaaja on hyväksynyt listan.

Tallenna työnäyte: Linkki GitHubin Issues-listaan sekä hyväksyjän rooli ja päivä viikon 35 päiväkirjaan.

Työvaihe 4 / 4 · Kokoa moodboard ja piirrä kaksi tekstuuriluonnosta.

- □ Valitse 5–8 värin paletti Lospecista tai kokoa oma. Tallenna paletista kuva nimellä paletti.png.
- □ Kerää 4–6 referenssikuvaa, jotka näyttävät teeman tunnelman. Moodboard menee julkiseen repositoryyn, joten käytä omia kuvia, omia pelikuvia tai CC0-kuvia, joissa ei näy ihmisiä, ja kirjaa lähteet.
- □ Kokoa paletti ja referenssit yhdeksi kuvaksi nimellä moodboard.png. Voit tehdä sen esimerkiksi Piskelissä, Canvassa tai paperille, jonka kuvaat.
- □ Piirrä kaksi karkeaa tekstuuriluonnosta paletin väreillä, paperille tai Piskeliin.
- □ Näytä moodboard ja luonnokset ohjaajalle. Kirjaa hänen kommenttinsa päiväkirjaan.
- □ Siirrä kuvat kansioon project-docs/evidence/week-35/, tee commit ja push.

Valmis kun: Kansiossa project-docs/evidence/week-35/ ovat paletti.png, moodboard.png ja kaksi luonnosta, ja ohjaajan kommentti on päiväkirjassa.

Tallenna työnäyte: Kuvat kansioon project-docs/evidence/week-35/, commit ja push. Polku viikon 35 päiväkirjaan.

Viikon lopputarkistus: Pakollinen sisältö (P0) on hyväksytty. Jokaisella issuella on arvio, prioriteetti ja Valmis kun -ehto. Moodboardissa näkyvät paletti ja referenssit, ja LICENSE-tiedosto on repositoryn juuressa.

Työnäyte Git-repositoryyn ennen rastia: asset-pack-suunnitelma.md, LICENSE-tiedosto, Issues-lista prioriteetteineen, rajauksen hyväksyntä sekä moodboard ja luonnokset.

Vaihe 2 – Paketin ilme ja ääni

Vko 36 · 31.8.–4.9. – Asset 1 · Ensimmäiset blokkitekstuurit peliin

Yhteys kokonaisprojektiin: Viikon 35 moodboard ja paletti lukitsivat ilmeen, ja viikon 34 tyhjä paketti on valmis ottamaan sisältöä. Nyt piirrät kolme blokkitekstuuria ja viet ne peliin ensimmäisellä kokonaisella asset-työkierrolla: luonnos, piirto, oikea polku, peli ja testi. Samalla kierrolla teet jokaisen tulevan assetin.

Viikon tavoite: Pelaaja näkee maailmassa kolme blokkia sinun omilla tekstuureillasi.

Piskel – piirrä pikselitekstuurit selaimessa

Lospec – väripaletit

Minecraft Wiki – resurssipaketin rakenne

Työvaihe 1 / 3 · Piirrä kolme blokkitekstuuria sovitulla paletilla.

- ☐ Valitse kolme pelin blokkia, joiden ilmeen teemasi muuttaa, esimerkiksi kivi, tammilankut ja multa.
- ☐ Etsi blokkien tiedostonimet Minecraft Wikistä, esimerkiksi stone.png. Kirjaa ne päiväkirjaan.
- ☐ Luo Piskelissä uusi 16 × 16 -kuva ja ota käyttöön moodboardin paletti.
- ☐ Piirrä ensimmäinen tekstuuri. Valo tulee ylhäältä: yläreuna vaaleampi, alareuna tummempi.
- ☐ Piirrä kaksi muuta tekstuuria samalla paletilla. Vie jokainen PNG-kuvana: Export → PNG.

Valmis kun: Sinulla on kolme 16 × 16 -kuvaa, jotka on piirretty moodboardin paletilla ja nimetty korvattavien blokkien mukaan.

Tallenna työnäyte: Luonnokset tai välivaiheiden kuvat kansioon project-docs/evidence/week-36/.

Työvaihe 2 / 3 · Vie tekstuurit pakettiin ja lataa paketti peliin.

- ☐ Luo repositoryn resourcepack-kansion sisään kansiopolku assets/minecraft/textures/block/.
- ☐ Siirrä kolme kuvaa sinne. Tiedostonimen pitää olla täsmälleen sama kuin korvattavalla blokilla. Iso ja pieni kirjain ovat eri asia.
- ☐ Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon. Korvaa vanha kansio. Paina pelissä F3 + T. Peli lataa paketin uudelleen.
- ☐ Luo testimaailma (Game Mode Creative, Allow Commands tai Allow Cheats ON) ja rakenna siihen jokaisesta kolmesta blokista oma 3 × 3 -ruudukko.

Valmis kun: Kolme omaa tekstuuria näkyy pelissä, eikä peli näytä virheilmoitusta.

Tallenna työnäyte: Commit ja push. Kuvakaappaus pelistä kansioon project-docs/evidence/week-36/.

Työvaihe 3 / 3 · Testaa saumat ja kirjaa testit.

- ☐ Kirjoita päiväkirjaan ennen testiä odotettu tulos: 3 × 3 -ruudukko näyttää yhtenäiseltä pinnalta.
- ☐ Testi 1: katso jokaista ruudukkoa läheltä. Kirjaa, näkyykö sauma tai häiritsevä toistuva kuvio.
- ☐ Testi 2: kävele 20 blokin päähän ja katso uudelleen. Kirjaa, erottuuko teema.
- ☐ Jos testi ei mennyt odotetusti, korjaa kuvaa muutamalla pikselillä ja vie se repositoryn kansioon. Kopioi resourcepack-kansio peliin, paina F3 + T ja testaa uudelleen. Kirjaa molemmat ajot.
- ☐ Tee commit ja push. Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Päiväkirjassa on kaksi testiä odotettuine ja todellisine tuloksineen, ja viimeinen ajo meni odotetusti.

Tallenna työnäyte: Testit ja commit-linkki viikon 36 päiväkirjaan.

Viikon lopputarkistus: Kolme omaa blokkitekstuuria näkyy pelissä ilman virheilmoituksia. Vierekkäisten blokkien saumat toimivat 3 × 3 -ruudukossa, ja tiedostot ovat Gitissä.

Työnäyte Git-repositoryyn ennen rastia: kuvakaappaus blokeista pelissä, tekstuurien commit ja kaksi testiä.

Vko 37 · 7.–11.9. – Asset 2 · Esinetekstuurit ja omat nimet

Yhteys kokonaisprojektiin: Esineet korvataan samalla periaatteella kuin viikon 36 blokit, vain kansio vaihtuu. Uutena kirjoitat ensimmäisen oman JSON-tiedoston, kielitiedoston, joka antaa sisällölle suomenkieliset nimet. Samaa JSON-tarkkuutta tarvitset viikolla 44 reseptissä ja saavutuksessa.

Viikon tavoite: Pelaaja näkee kaksi omaa esinettä, ja muokattu sisältö näkyy teemasi mukaisilla suomenkielisillä nimillä.

Työvaihe 1 / 3 · Piirrä kaksi esinetekstuuria ja vie ne pakettiin.

- □ Valitse kaksi esinettä, jotka sopivat teemaasi, esimerkiksi leipä ja rautamiekka. Etsi niiden tiedostonimet Minecraft Wikistä.
- □ Piirrä Piskelissä molemmat 16 × 16 -kokoon moodboardin paletilla. Jätä tausta läpinäkyväksi ja väritä vain esine.
- □ Luo resourcepack-kansioon polku assets/minecraft/textures/item/ ja siirrä kuvat sinne korvattavan esineen nimellä.
- □ Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon. Korvaa vanha kansio. Paina pelissä F3 + T. Peli lataa paketin uudelleen.
- □ Tarkista esine tavaraluettelossa. Ota se sitten käteen ja katso sitä.

Valmis kun: Kaksi omaa esinetekstuuria näkyy pelissä tavaraluettelossa ja kädessä.

Tallenna työnäyte: Commit ja push. Välivaiheiden kuvat ja kuvakaappaus pelistä kansioon project-docs/evidence/week-37/.

Työvaihe 2 / 3 · Anna sisällölle omat suomenkieliset nimet kielitiedostolla.

- □ Luo VS Codella tiedosto resourcepack/assets/minecraft/lang/fi_fi.json.
- □ Paina pelissä F3 + H. Nyt esineen tunnus näkyy esineen kuvauksessa, esimerkiksi minecraft:bread.
- □ Kirjoita tiedostoon jokaiselle muokkaamallesi blokille ja esineelle rivi avun mallin mukaan. Avain on esimerkiksi block.minecraft.stone, ja arvo on antamasi uusi nimi.
- □ Vaihda pelin kieleksi suomi (Options → Language). Valikot ovat sen jälkeen suomeksi, esimerkiksi Options = Asetukset. Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon. Korvaa vanha kansio. Paina pelissä F3 + T. Peli lataa paketin uudelleen.
- □ Tarkista tavaraluettelosta, että omat nimet näkyvät.

Valmis kun: Muokatut blokit ja esineet näkyvät pelissä omilla suomenkielisillä nimillä.

Tallenna työnäyte: Commit ja push. Kuvakaappaus nimistä kansioon project-docs/evidence/week-37/.

Työvaihe 3 / 3 · Riko kielitiedosto tarkoituksella ja testaa nimet pelissä.

- □ Kirjoita päiväkirjaan ennen testiä odotettu tulos: rikkinäisellä tiedostolla omat nimet katoavat.
- □ Poista fi_fi.json-tiedostosta yksi pilkku. Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon. Korvaa vanha kansio. Paina pelissä F3 + T. Peli lataa paketin uudelleen.
- □ Kirjaa, mitä pelissä tapahtui.
- □ Palauta pilkku. Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon ja korvaa vanha, paina F3 + T ja kirjaa, palasivatko omat nimet.
- □ Tee commit ja push. Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Rikkinäisen ja korjatun tiedoston tulokset on kirjattu, ja omat nimet näkyvät taas pelissä.

Tallenna työnäyte: Testin tulokset ja commit-linkki viikon 37 päiväkirjaan.

Viikon lopputarkistus: Esineet näkyvät omilla tekstuureilla. Muokatut blokit ja esineet näkyvät suomenkielisillä nimillä. Rikkinäisen JSONin vaikutus on testattu ja kirjattu.

Työnäyte Git-repositoryyn ennen rastia: kuvakaappaus esineistä ja nimistä, fi_fi.json-commit ja testi rikkinäisellä JSONilla.

Vko 38 · 14.–18.9. – Asset 3 · Ensimmäinen Blockbench-malli

Yhteys kokonaisprojektiin: Viikoilla 36 ja 37 muutit blokkien ja esineiden pinnan, nyt muutat muodon samalla korvausperiaatteella. Rakennat Blockbenchissä pienen mallin, teksturoit sen ja korvaat sillä yhden blokin mallin. Pieni malli opettaa työkalun ennen viikon 39 isompaa työtä.

Viikon tavoite: Pelaaja voi asettaa maahan blokin, jolla on sinun Blockbenchissä tekemäsi 3D-muoto.

Blockbench – lataa tai käytä selaimessa

Blockbench Wiki – aloitusohjeet

Työvaihe 1 / 3 · Mallinna Blockbenchillä pieni blokkimalli.

- □ Valitse koristeblokki, jonka muoto saa muuttua, esimerkiksi kukkaruukku (flower_pot). Kirjaa valinta päiväkirjaan.
- □ Avaa Blockbench ja valitse File → New → Java Block/Item. Anna projektille teeman mukainen nimi.
- □ Rakenna muoto Add Cube -painikkeella 2–4 kuutiosta. Pysy 16 × 16 × 16 -ruudukon sisällä, niin blokki istuu maailmaan.
- □ Tallenna Blockbench-projekti: File → Save Project. Siirrä .bbmodel-tiedosto kansioon project-docs/lahdetiedostot/.

Valmis kun: Blockbenchissä on 2–4 kuution malli, joka mahtuu yhden blokin kokoon, ja .bbmodel-tiedosto on tallennettu.

Tallenna työnäyte: Commit ja push. Kuvakaappaus Blockbenchistä kansioon project-docs/evidence/week-38/.

Työvaihe 2 / 3 · Teksturoi malli ja vie se pakettiin.

- □ Luo tekstuuri Blockbenchin Textures-paneelissa: Create Texture, rastita Template ja paina Confirm. Siirry sitten Paint-tilaan ja maalaa mallin pinnat moodboardin paletilla. Blockbench tekee UV-kartan eli pintojen ja kuvan vastaavuuden puolestasi.
- □ Tallenna tekstuuri polkuun resourcepack/assets/minecraft/textures/block/ omalla nimellä, esimerkiksi oma_koriste.png.
- □ Vie malli: File → Export → Java Block/Item Model. Tallenna se polkuun resourcepack/assets/minecraft/models/block/ korvattavan blokin nimellä, esimerkiksi flower_pot.json.
- □ Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon. Korvaa vanha kansio. Paina pelissä F3 + T. Peli lataa paketin uudelleen.

Valmis kun: Malli ja tekstuuri ovat oikeissa kansioissa, ja peli lataa paketin ilman virheilmoitusta.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 38 päiväkirjaan.

Työvaihe 3 / 3 · Testaa malli pelissä eri suunnista.

- □ Kirjoita päiväkirjaan ennen testiä odotettu tulos: malli näkyy oikein joka suunnasta, eikä pinnoissa ole violettimustaa ruutukuviota.
- □ Aseta blokki maahan ja katso sitä neljästä ilmansuunnasta. Ota kuvakaappaus.
- □ Ota blokki käteen. Jos esine näkyy kädessä litteänä kuvana, se on normaalia: monella blokilla on oma kuva esineenä. Kirjaa, mitä näit.
- □ Jos jollakin pinnalla näkyy violettimusta ruutukuvio, peli ei löydä tekstuuria. Tarkista tekstuurin nimi ja polku ja korjaa. Kopioi resourcepack-kansio uudelleen peliin, paina F3 + T ja testaa uudelleen.
- □ Tee commit ja push. Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Päiväkirjassa on testin odotettu ja todellinen tulos, ja malli näkyy pelissä oikein joka suunnasta.

Tallenna työnäyte: Kuvakaappaukset kansioon project-docs/evidence/week-38/ ja testin tulos viikon 38 päiväkirjaan.

Viikon lopputarkistus: Oma malli näkyy pelissä oikein maassa, kädessä ja eri suunnista katsottuna ilman virheilmoituksia. Malli ja tekstuuri ovat Gitissä.

Työnäyte Git-repositoryyn ennen rastia: Blockbench-projektitiedosto, kuvakaappaukset mallista pelissä ja commit.

Vko 39 · 21.–25.9. – Asset 4 · Isompi malli tai hahmon uusi ilme

Yhteys kokonaisprojektiin: Viikon 38 pieni malli opetti Blockbenchin ja mallin viennin pakettiin. Nyt vertaat kahta isompaa vaihtoehtoa ja toteutat perustellusti toisen, koska vertailu ja päätös ovat osa näyttöä. Valmis työ on paketin näyttävin asset, jonka testaajat näkevät viikolla 41.

Viikon tavoite: Paketin näyttävin asset, isompi malli tai mobin uusi ilme, näkyy pelissä läheltä, kaukaa ja yöllä.

Työvaihe 1 / 3 · Vertaa kahta vaihtoehtoa ja tee valinta.

- □ Lue kaksi vaihtoehtoa: A = isompi blokkimalli, B = mobin eli pelin hahmon uusi tekstuuri, esimerkiksi creeperin uusi ilme.
- □ Pyydä ohjaajaa tai vertaista eli toista opiskelijaa keskustelemaan kanssasi kymmeneksi minuutiksi.
- □ Arvioikaa kumpaakin vaihtoehtoa kolmella kysymyksellä: montako päivää työ vie, kuinka usein se näkyy pelissä ja mikä voi mennä pieleen?
- □ Valitse A tai B. Kirjoita päiväkirjaan valinta, kaksi perustelua ja keskustelukumppanin rooli.

Valmis kun: Päiväkirjassa ovat molemmat vaihtoehdot, kolmen kysymyksen vastaukset, valinta, kaksi perustelua ja keskustelukumppanin rooli.

Tallenna työnäyte: Vertailu viikon 39 päiväkirjaan.

Työvaihe 2 / 3 · Toteuta valittu työ kahdessa vaiheessa.

- □ Päivä 1: tee runko. Vaihtoehdossa A rakenna muoto Blockbenchissä. Vaihtoehdossa B avaa mobin tekstuuripohja Blockbenchissä ja piirrä pääväritys.
- □ Tee commit ja push viestillä ”Runko”.
- □ Päivä 2: piirrä yksityiskohdat samalla paletilla kuin muut assetit.
- □ Vie työ pakettiin. Vaihtoehdossa A vie malli kuten viikolla 38. Vaihtoehdossa B vie tekstuuri polkuun resourcepack/assets/minecraft/textures/entity/, esimerkiksi creeper/creeper.png.
- □ Tarkista mobin tekstuurin polku versiosi mukaan Minecraft Wikistä: esimerkiksi lehmän ja sian tiedostonimet muuttuivat versiossa 1.21.5.
- □ Tee commit ja push viestillä ”Yksityiskohdat”.

Valmis kun: Valittu työ on paketissa, ja repositoryssa on kaksi erillistä committia: runko ja yksityiskohdat.

Tallenna työnäyte: Kahden commitin linkit viikon 39 päiväkirjaan.

Työvaihe 3 / 3 · Testaa työ läheltä, kaukaa ja yöllä.

- □ Kirjoita päiväkirjaan ennen testiä odotettu tulos: teema tunnistuu läheltä, kaukaa ja yöllä.
- □ Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon. Korvaa vanha kansio. Paina pelissä F3 + T. Peli lataa paketin uudelleen.
- □ Etsi tai kutsu muokattu kohde pelissä. Ota kuvakaappaus läheltä, 20 blokin päästä ja yöllä.
- □ Kirjaa jokaisesta kuvasta, tunnistuuko teema. Jos ei tunnistu, korjaa, kopioi resourcepack-kansio uudelleen peliin, paina F3 + T ja testaa uudelleen.
- □ Tee commit ja push. Kirjoita GitHubissa jokaiseen tämän viikon valmiiseen issueen kommentti. Lisää kommenttiin linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Kolme kuvaa ja testin tulos on kirjattu, ja teema tunnistuu jokaisesta kuvasta.

Tallenna työnäyte: Kuvat kansioon project-docs/evidence/week-39/ ja testin tulos viikon 39 päiväkirjaan.

Viikon lopputarkistus: Valittu kokonaisuus toimii pelissä ja näyttää hyvältä läheltä, kaukaa ja pimeässä. Vertailumuistio päätöksineen on kirjattu.

Työnäyte Git-repositoryyn ennen rastia: vertailu ja valinta päiväkirjassa, kaksi toteutuksen committia ja kuvat pelistä.

Vko 40 · 28.9.–2.10. – Asset 5 · Äänet ja tunnelma

Yhteys kokonaisprojektiin: Paketti näyttää jo teemaltasi, ja nyt se saa äänen. Lisäät äänen paketin omaan nimiavaruuteen ja kirjaat sen lähteen ja lisenssin CREDITS-tiedostoon, koska avoimessa julkaisussa luvaton ääni estää julkaisun. Tämän viikon jälkeen resurssipaketti on koossa, ja viikolla 41 testaajat kokeilevat sitä.

Viikon tavoite: Pelaaja kuulee paketin oman äänen ja näkee sen tekstityksen, ja äänen lisenssi sallii julkaisun.

Freesound – CC-lisensoituja ääniä, tarkista lisenssi

Minecraft Wiki – sounds.json

Työvaihe 1 / 3 · Valitse ääni ja kirjaa sen lisenssi CREDITS-tiedostoon.

- □ Hanki ääni. Äänitä puhelimella lyhyt teemaan sopiva ääni, jossa ei kuulu kenenkään puhetta, tai etsi ääni Freesoundista. Freesoundin lataus vaatii tilin: sovi tilistä ohjaajan kanssa.
- □ Tarkista lisenssi. Jos ääni ei ole oma, tarkista sen lisenssi. Kelpaa vain CC0 tai CC BY. Jos lisenssissä on NC (ei kaupalliseen käyttöön) tai ND (ei muokkauksia), valitse toinen ääni.
- □ Luo CREDITS.md. Luo repositoryn juureen tiedosto CREDITS.md, jos sitä ei vielä ole. CREDITS on luettelo pakettiin tuoduista lähteistä.
- □ Kirjaa äänen lähde. Kirjoita CREDITS.md-tiedostoon äänen nimi, tekijä, linkki ja lisenssi. Jos äänitit itse, kirjoita ”oma äänitys” ja päivä.
- □ Tallenna Gitiin. Tee commit ja push.

Valmis kun: CREDITS.md-tiedostossa on äänen tekijä, lähde ja lisenssi, ja lisenssi sallii uudelleenjulkaisun.

Tallenna työnäyte: Linkki CREDITS.md-tiedostoon viikon 40 päiväkirjaan.

Työvaihe 2 / 3 · Muunna ääni ogg-muotoon ja lisää se sounds.json-tiedostoon.

- □ Asenna Audacity. Asenna Audacity, jos sitä ei vielä ole koneella. Koulun koneella asennuksen voi joutua tekemään ohjaaja.
- □ Leikkaa ja vie ogg-muotoon. Leikkaa ääni Audacityssä enintään kahden sekunnin mittaiseksi. Vie se: File → Export Audio, muodoksi Ogg Vorbis, ei Opus (vanhemmassa Audacityssä File → Export → Export as OGG).
- □ Nimeä tiedosto. Käytä pieniä kirjaimia ilman ääkkösiä ja välilyöntejä, esimerkiksi kyla_kello.ogg.
- □ Luo oma nimiavaruus. Luo resourcepack-kansioon kansio assets/teema/sounds/ ja siirrä ääni sinne. Vaihda sana teema oman pakettisi nimeksi pienillä kirjaimilla ilman ääkkösiä, esimerkiksi kotikyla.
- □ Kirjoita sounds.json. Kirjoita tiedosto assets/teema/sounds.json avun mallin mukaan.
- □ Tallenna Gitiin. Tee commit ja push.

Valmis kun: Ogg-tiedosto ja sounds.json ovat oman nimiavaruuden kansiossa.

Tallenna työnäyte: Commit-linkki viikon 40 päiväkirjaan.

Työvaihe 3 / 3 · Testaa ääni pelissä /playsound-komennolla.

- □ Kirjaa odotus. Kirjoita päiväkirjaan ennen testiä odotettu tulos: ääni kuuluu ja tekstitys näkyy.
- □ Lataa paketti peliin. Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon ja korvaa vanha kansio. Paina pelissä F3 + T, niin peli lataa paketin uudelleen.
- □ Kytke tekstitykset. Valitse Options → Accessibility Settings → Show Subtitles: ON. Suomenkielisessä pelissä sama kohta on Asetukset-valikossa.
- □ Soita ääni komennolla. Kirjoita chattiin /playsound teema:kyla.kello master @s. Vaihda sana teema ja äänen nimi omiksi.
- □ Varmista komentojen lupa. Jos komento ei toimi, avaa Esc → Open to LAN, laita Allow Commands (tai Allow Cheats) päälle ja paina Start LAN World. Jos komento toimi, rastita tämä suoraan.
- □ Kirjaa tulos ja säädä. Kirjaa tulos. Jos ääni on liian kova tai hiljainen, säädä sitä Audacityssä, vie ogg uudelleen repositoryn kansioon ja lataa paketti peliin ennen uutta testiä.
- □ Kuittaa issuet. Tee commit ja push. Kirjoita jokaiseen tämän viikon valmiiseen GitHub-issueen kommentti, jossa on linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Ääni kuuluu /playsound-komennolla, tekstitys näkyy, ja testin tulos on päiväkirjassa.

Tallenna työnäyte: Kuvakaappaus tekstityksestä kansioon project-docs/evidence/week-40/. Testin tulos, kuvan polku ja commit-linkki viikon 40 päiväkirjaan.

Viikon lopputarkistus: Oma ääni kuuluu pelissä /playsound-komennolla, tekstitys näkyy ja äänen lähde sekä lisenssi on kirjattu CREDITS-tiedostoon.

Työnäyte Git-repositoryyn ennen rastia: CREDITS.md-kirjaus, sounds.json-commit ja /playsound-testin tulos.

Vaihe 3 – Palaute ja pelillinen lisä

Vko 41 · 5.–9.10. – Katselmointi · Ensimmäiset testaajat

Yhteys kokonaisprojektiin: Viikoilla 36–40 rakensit resurssipaketin, jossa on tekstuurit, nimet, mallit ja ääni. Nyt testaajat kokeilevat väliversiota, koska toimeksiantaja haluaa nähdä toimivan version ennen lopullista ja omalle työlle sokeutuu. Palautteesta sovittu muutos tehdään loman jälkeen viikolla 43.

Viikon tavoite: Ohjaaja ja vertainen asentavat väliversion ohjeesi avulla, ja heidän palautteestaan sovitaan yksi muutos.

Työvaihe 1 / 3 · Kokoa testaajille väliversio ja asennusohje.

- □ Pakkaa väliversio. Avaa repositoryn resourcepack-kansio ja valitse sen sisältö: pack.mcmeta, pack.png ja assets. Pakkaa ne zipiksi. Älä pakkaa itse kansiota, muuten peli ei tunnista pakettia.
- □ Tarkista zip. Avaa zip ja tarkista, että ensimmäisellä tasolla on pack.mcmeta eikä kansio.
- □ Kirjoita asennusohje. Kirjoita ohje avun pohjan mukaan tiedostoon project-docs/evidence/week-41/asennusohje.md.
- □ Kokeile puhtaassa pelissä. Tee Minecraft Launcherissa Installations → New installation: sama versio ja Game Directory -kohtaan uusi tyhjä kansio. Asenna zip siihen ohjeen avulla ja korjaa ohje, jos poikkesit siitä.
- □ Sovi katselmointi. Sovi ohjaajan kanssa katselmoinnin aika. Katselmointi on tapaaminen, jossa testaajat kokeilevat keskeneräistä pakettia ja antavat palautetta.
- □ Tallenna Gitiin. Tee commit ja push.

Valmis kun: Väliversion zip asentuu pelkän ohjeen avulla, ja katselmoinnin aika on sovittu.

Tallenna työnäyte: Zip ja asennusohje kansioon project-docs/evidence/week-41/, commit ja push.

Työvaihe 2 / 3 · Anna testaajien kokeilla ja kirjaa, mitä he sanovat.

- □ Pyydä testaajat. Testaajat ovat ohjaaja ja yksi vertainen eli toinen opiskelija. Pyydä heitä asentamaan paketti ohjeesi avulla ja pelaamaan vapaasti.
- □ Kirjaa havainnot roolin mukaan. Älä neuvo, ellei testaaja pyydä apua. Kirjoita ylös, mitä testaaja sanoo ja tekee, roolin mukaan: ohjaaja tai testaaja A. Älä kirjoita nimiä.
- □ Kirjaa omat havainnot. Kirjoita erikseen, mitä itse näit: mitä testaaja huomasi ja mitä hän ohitti.
- □ Esittele valintasi. Kerro testauksen jälkeen viidessä minuutissa paketin sisältö ja kolme omaa valintaasi perusteluineen.
- □ Kysy tärkein muutos. Kysy lopuksi: mikä yksi asia pitäisi muuttaa ensin?

Valmis kun: Muistiinpanoissa ovat testaajien havainnot roolin mukaan ja omat havaintosi erikseen sekä kolme selittämääsi valintaa.

Tallenna työnäyte: Muistiinpanot viikon 41 päiväkirjaan roolein, ilman nimiä. Jos ohjaaja tarvitsee nimen tai sanatarkan lausuman, lähetä se hänelle Teamsissa.

Työvaihe 3 / 3 · Sovi testaajien palautteesta yksi muutos.

- □ Valitse muutos. Valitse palautteesta yhdessä ohjaajan kanssa yksi muutos, jonka ehdit tehdä viikolla 43.
- □ Tee muutosissue. Tee muutoksesta GitHub-issue: verbillä alkava otsikko, testaajan havainto roolin kanssa (esimerkiksi testaaja A), arvio ja Valmis kun -ehto.
- □ Anna prioriteetti. Jos muutos on välttämätön, se on pakollinen (P0): anna label P0 pakollinen. Muuten se on tärkeä (P1): anna label P1 tärkeä.
- □ Vertaa arvioita toteumaan. Valitse viikolla 35 tehdyistä issueista kolme. Vertaa niiden arviota siihen, kauanko työ oikeasti kesti. Kirjaa ero ja sen syy.
- □ Kirjoita katselmointimuistio. Kirjaa päiväkirjaan päivä, osallistujien roolit, testaajien havainnot roolin mukaan, oma tulkinta, päätös ja hyväksyjä.

Valmis kun: GitHubissa on yksi hyväksytty muutosissue, ja katselmointimuistio sekä arvioiden vertailu ovat päiväkirjassa.

Tallenna työnäyte: Issuen linkki ja katselmointimuistio viikon 41 päiväkirjaan.

Viikon lopputarkistus: Ohjaaja ja vertaistestaaja ovat kokeilleet pakettia pelissä. Muistiossa näkyvät testaajien havainnot roolin mukaan ilman nimiä, oma tulkinta, päätös, hyväksyjä ja yksi rajattu issue.

Työnäyte Git-repositoryyn ennen rastia: väliversion zip ja asennusohje, katselmointimuistio, palaute ja hyväksytty muutosissue.

Vko 42 · 12.–16.10. – Syysloma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 43 viimeisimmästä toimivasta main-versiosta.

Vko 43 · 19.–23.10. – Asset 6 · Palautemuutos ja datapaketin runko

Yhteys kokonaisprojektiin: Viikon 41 katselmoinnista sovittiin yksi muutos, ja sen issue odottaa. Teet muutoksen omassa haarassa ja yhdistät sen pull requestilla, jotta main-haara pysyy koko ajan toimivana. Samalla paketti saa toisen puoliskonsa, datapaketin rungon, johon viikon 44 resepti ja saavutus rakennetaan.

Viikon tavoite: Testaajan pyytämä muutos näkyy pelissä, ja datapaketti tervehtii pelaajaa chatissa, kun maailma latautuu.

Työvaihe 1 / 3 · Tee palautemuutos omassa Git-haarassa.

- □ Kirjoita hyväksymistesti. Avaa viikolla 41 tekemäsi muutosissue ja kirjoita siihen, mitä testaaja näkee pelissä, kun muutos on valmis.
- □ Luo haara. Luo uusi haara eli branch: GitHub Desktopissa Current Branch → New Branch. Anna nimeksi esimerkiksi korjaus/lyhdyn-hehku.
- □ Tee muutos pienissä osissa. Tee commit aina, kun yksi osa toimii.
- □ Testaa pelissä. Kopioi repositoryn resourcepack-kansio pelin resourcepacks-kansioon ja korvaa vanha kansio. Paina F3 + T ja testaa muutos pelissä.
- □ Julkaise haara. Tee push. Uuden haaran ensimmäisessä pushissa GitHub Desktopin painike on nimeltään Publish branch.

Valmis kun: Haara näkyy GitHubissa, ja siinä on vähintään kaksi pientä committia.

Tallenna työnäyte: Haaran nimi ja commit-linkit viikon 43 päiväkirjaan.

Työvaihe 2 / 3 · Yhdistä muutos main-haaraan pull requestilla.

- □ Avaa pull request. Paina GitHubissa Compare & pull request. Jos painiketta ei näy, valitse Pull requests → New pull request ja compare-kohtaan oma haarasi. Pull request on pyyntö yhdistää haara main-haaraan.
- □ Kuvaa muutos ja testi. Kirjoita pull requestiin, mitä muutit ja miten testasit. Lisää rivi Closes #numero, jossa numero on muutosissuen numero.
- □ Pyydä kommentti. Pyydä ohjaajaa tai vertaista kommentoimaan. Vastaa kommenttiin tai tee korjaus samaan haaraan ja tee push.
- □ Yhdistä main-haaraan. Paina Merge pull request ja Confirm merge. Yhdistämistä kutsutaan mergeksi.
- □ Päivitä oma main. Vaihda GitHub Desktopissa haaraksi main ja paina Fetch origin ja Pull origin, jotta koneesi main-haara on ajan tasalla.

Valmis kun: Pull request on yhdistetty, siinä on kommentti ja vastaus, ja muutosissue on suljettu.

Tallenna työnäyte: Pull requestin linkki viikon 43 päiväkirjaan.

Työvaihe 3 / 3 · Luo datapaketin runko ja ensimmäinen funktio.

- □ Tee pack.mcmeta. Tee repositoryn datapack-kansioon pack.mcmeta avun mallin mukaan. Tarkista datapaketin pack_format-arvo samasta wikitaulukosta: se on eri kuin resurssipaketilla.
- □ Kirjoita load.mcfunction. Luo kansiot data/teema/function/ ja kirjoita sinne load.mcfunction: yksi tellraw-komento, joka tulostaa tervehdyksen chattiin. Käytä samaa nimiavaruutta kuin äänissä, esimerkiksi kotikyla.
- □ Kirjoita load.json. Kirjoita tiedosto data/minecraft/tags/function/load.json. Se kertoo pelille, mikä funktio ajetaan, kun datapaketti latautuu.
- □ Kopioi testimaailmaan. Luo maailma: Game Mode Creative, Allow Commands (tai Allow Cheats) ON. Poistu Save and Quit to Title, kopioi datapack-kansio maailman datapacks-kansioon (Edit → Open World Folder) ja avaa maailma.
- □ Tarkista lataus. Aja pelissä /reload ja /datapack list. Vihreä tervehdys chatissa ja paketti listassa tarkoittavat, että runko toimii.
- □ Tallenna Gitiin. Tee commit ja push.

Valmis kun: /reload näyttää tervehdyksen, ja /datapack list näyttää paketin ilman virheitä.

Tallenna työnäyte: Kuvakaappaus chatista kansioon project-docs/evidence/week-43/. Kuvan polku ja commit-linkki viikon 43 päiväkirjaan.

Viikon lopputarkistus: Main-haarassa on testattu palautemuutos. Datapaketti latautuu /reload-komennolla ilman virheitä, ja funktio toimii /function-komennolla.

Työnäyte Git-repositoryyn ennen rastia: muutosissue, haara, pull request kommentteineen ja datapaketin rungon commit.

Vko 44 · 26.–30.10. – Asset 7 · Skriptattu ominaisuus

Yhteys kokonaisprojektiin: Viikon 43 datapaketin runko latautuu ja ajaa funktion, mutta pelillistä sisältöä siinä ei vielä ole. Nyt lisäät reseptin, saavutuksen ja palkintofunktion, jotka muodostavat ketjun: valmistus laukaisee saavutuksen, ja saavutus ajaa palkinnon. Kun ketju toimii, paketin koko pakollinen sisältö on valmis ja viikolla 45 testataan kokonaisuus.

Viikon tavoite: Pelaaja valmistaa teemasi reseptin selviytymistilassa ja saa siitä saavutuksen ja palkinnon.

Misode – reseptigeneraattori, tarkista rakenne

Misode – advancement-generaattori

Työvaihe 1 / 3 · Tee teemaasi sopiva valmistusresepti.

- □ Päätä resepti. Päätä teemaasi sopivat ainekset ja tulos. Valitse ainekset, jotka saa selviytymistilassa noin 15 minuutissa, koska testaat reseptin niin työvaiheessa 3. Kirjaa päätös päiväkirjaan.
- □ Luo reseptitiedosto. Luo datapack-kansioon polku data/teema/recipe/ ja sinne tiedosto teemalyhty.json avun mallin pohjalta. Vaihda nimi, ainekset ja tulos omiksi.
- □ Tarkista rakenne. Kohta pattern kuvaa 3 × 3 -työpöydän riveittäin, ja key kertoo, mitä kukin kirjain tarkoittaa. Tarkista rakenne sovitun version mukaan Minecraft Wikistä tai Misoden reseptigeneraattorista (valitse ensin peliversio).
- □ Lataa datapaketti. Kopioi repositoryn datapack-kansio testimaailman datapacks-kansioon ja korvaa vanha. Aja /reload.
- □ Anna resepti itsellesi. Aja /recipe give @s teema:teemalyhty ja tarkista työpöydän reseptikirjasta, että resepti löytyy. Datapaketin resepti näkyy kirjassa vasta, kun pelaaja on saanut sen.

Valmis kun: Oma resepti näkyy reseptikirjassa /recipe give -komennon jälkeen, ja työpöytä antaa tuloksen, kun ainekset ovat oikeassa kuviossa.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 44 päiväkirjaan.

Työvaihe 2 / 3 · Tee saavutus, joka palkitsee pelaajan funktiolla.

- □ Kirjoita palkintofunktio. Kirjoita data/teema/function/palkinto.mcfunction: yksi tellraw-viesti ja pieni xp-palkinto riittävät.
- □ Luo saavutus. Luo tiedosto data/teema/advancement/kylan_valot.json avun mallin pohjalta.
- □ Tunnista kolme osaa. Tarkista mallista: display näyttää saavutuksen, criteria kertoo, milloin se laukeaa, ja rewards ajaa palkintofunktion.
- □ Kytke oma resepti. Vaihda laukaisimen recipe_id oman reseptisi nimeksi, esimerkiksi teema:teemalyhty.
- □ Testaa palkinto. Kopioi datapack-kansio testimaailmaan ja korvaa vanha. Aja /reload ja /function teema:palkinto: viesti ja xp tulevat, jos funktio latautui. JSON-virheet näkyvät vain lokissa, eivät chatissa.

Valmis kun: /function teema:palkinto antaa viestin ja xp:n, ja kylan_valot näkyy ehdotuksena, kun kirjoitat chattiin /advancement grant @s only teema: (älä paina Enter).

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 44 päiväkirjaan.

Työvaihe 3 / 3 · Testaa koko polku uudessa selviytymismaailmassa.

- □ Kirjaa odotus. Kirjoita päiväkirjaan ennen testiä odotettu tulos: kun valmistat reseptin, saavutus ja palkintoviesti ilmestyvät.
- □ Luo selviytymismaailma. Game Mode Survival, Allow Commands (tai Allow Cheats) ON nollausta varten. Poistu Save and Quit to Title, kopioi datapack-kansio maailman datapacks-kansioon (Edit → Open World Folder) ja avaa maailma.
- □ Pelaa ainekset. Hanki ainekset pelaamalla, ilman komentoja. Valmista esine työpöydällä.
- □ Kirjaa tulos. Kirjaa, laukesiko saavutus ja tuliko palkinto. Uusintatestiä varten saavutuksen voi nollata komennolla /advancement revoke @s only teema:kylan_valot.
- □ Tallenna kuvat. Tallenna kuvasarja tai lyhyt video koko polusta. Saavutusilmoitus näyttää chatissa pelaajanimesi: peitä se kuvista, ellei nimeä ole sovittu julkiseksi.
- □ Kuittaa issuet. Tee commit ja push. Kirjoita jokaiseen tämän viikon valmiiseen GitHub-issueen kommentti, jossa on linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Koko polku ainesten hankinnasta palkintoon toimii selviytymistilassa, ja testin tulos on kirjattu.

Tallenna työnäyte: Kuvat tai video kansioon project-docs/evidence/week-44/ ja testin tulos viikon 44 päiväkirjaan.

Viikon lopputarkistus: Resepti löytyy reseptikirjasta ja tuottaa esineen. Saavutus laukeaa reseptin valmistuksesta, ja palkintofunktio toimii. Koko polku on testattu uudessa selviytymismaailmassa.

Työnäyte Git-repositoryyn ennen rastia: reseptin ja saavutuksen commitit sekä kuvat tai video koko polusta selviytymistilassa.

Vaihe 4 – Testaus ja laatu

Vko 45 · 2.–6.11. – Paketin laatu · Paketti kestää käyttöä

Yhteys kokonaisprojektiin: Kaikki pakollinen sisältö on nyt paketissa, mutta sitä on testattu vain osa kerrallaan. Nyt kirjoitat 12 testitapausta ennen ajoa, ajat ne puhtaassa pelissä ja korjaat kolme virhettä koko ketjuna. Samoja testitapauksia käytät viikoilla 47 ja 48 regressiotesteinä.

Viikon tavoite: Paketti on testattu 12 kirjatulla testitapauksella, ja kolme löydettyä virhettä on korjattu ja testattu uudelleen.

Avaa näyttöaineisto

Työvaihe 1 / 3 · Kirjoita 12 testitapausta ennen testaamista.

- ☐ Luo testaus.md. Luo tiedosto project-docs/testaus.md avun taulukkopohjasta.
- ☐ Numeroi testitapaukset. Numeroi ne T01, T02, T03 ja niin edelleen. T tarkoittaa testitapausta ja numero sen järjestystä.
- ☐ Kirjoita tavallinen käyttö. Kirjoita neljä testitapausta: asennus ohjeella, blokkitekstuurit, malli ja omat nimet.
- ☐ Kirjoita rajatapaukset. Kirjoita neljä testitapausta: tekstitykset päällä, englanninkielinen peli, yö ja resepti selviytymistilassa.
- ☐ Kirjoita virhetilanteet. Kirjoita neljä testitapausta: väärä pack_format, puuttuva tekstuuri, rikkinäinen JSON ja datapaketti ilman /reload-komentoa.
- ☐ Kirjoita odotukset. Täytä jokaiseen testitapaukseen lähtötila, mitä teet, ja odotettu tulos. Älä aja vielä yhtään testiä.

Valmis kun: Tiedostossa on 12 testitapausta T01–T12, ja jokaisella on odotettu tulos.

Tallenna työnäyte: project-docs/testaus.md, commit ja push.

Työvaihe 2 / 3 · Aja testitapaukset puhtaassa pelissä ja kirjaa tulokset.

- ☐ Tee puhdas asennus. Tee Minecraft Launcherissa uusi asennus: Installations → New installation. Valitse sovittu versio ja Game Directory -kohtaan uusi tyhjä kansio, jotta vanhat paketit eivät näy.
- ☐ Aja testitapaukset. Aja testitapaukset T01–T12 järjestyksessä. Kirjaa jokaisesta heti, mitä tapahtui ja läpäisikö testi.
- ☐ Tee virhetestit. Muuta repositoryssa pack_format vääräksi, poista yksi tekstuuri tai riko JSON ja kopioi kansio puhtaan pelin kansioon. Palauta tiedosto testin jälkeen GitHub Desktopissa: hiiren oikea → Discard changes.
- ☐ Lue loki. Avaa virhetestin jälkeen asennuksen kansiosta tiedosto logs/latest.log VS Codella ja etsi sanoja ERROR ja WARN. Kirjaa, mitä loki kertoi virheestä.
- ☐ Merkitse vain ajetut. Älä merkitse testiä läpäistyksi, jos et ajanut sitä itse.

Valmis kun: Jokaisella testitapauksella T01–T12 on todellinen tulos ja merkintä, läpäisikö se.

Tallenna työnäyte: Päivitetty project-docs/testaus.md ja virhetestien kuvakaappaukset sekä lokiote kansioon project-docs/evidence/week-45/, commit ja push.

Työvaihe 3 / 3 · Korjaa kolme virhettä ja kirjaa jokaisesta koko ketju.

- ☐ Valitse kolme virhettä. Valitse kolme testitapausta, jotka eivät läpäisseet. Jos aitoja virheitä ei ole kolmea, pyydä ohjaajalta vikatehtävä eli valmisteltu virhe, jonka etsit ja korjaat.
- ☐ Kirjaa toistaminen. Kirjaa jokaisesta, miten virhe toistetaan, mikä oli odotettu tulos ja mitä tapahtui.
- ☐ Etsi syy. Etsi syy ennen kuin korjaat. Kirjoita syy yhdellä virkkeellä.
- ☐ Korjaa ja tee commit. Commitin viestissä on testitapauksen tunnus, esimerkiksi ”Korjaa testitapaus T09: oikea pack_format”.
- ☐ Aja uusinta- ja regressiotesti. Lataa korjattu kansio peliin (F3 + T tai /reload) ja aja sama testitapaus uudelleen. Aja myös regressiotesti: toinen samaa tiedostoa koskeva testitapaus.
- ☐ Kirjaa ketju. Kirjaa jokainen ketju päiväkirjaan: havainto → toistamisohje → syy → korjauscommit → uusintatesti → regressiotesti.

Valmis kun: Päiväkirjassa on kolme täydellistä ketjua, ja jokaisen uusintatesti meni odotetusti.

Tallenna työnäyte: Kolme ketjua commit-linkkeineen viikon 45 päiväkirjaan.

Viikon lopputarkistus: Kaikissa 12 testitapauksessa näkyvät lähtötila, toiminta, odotus, havainto ja tulos. Kolmessa ketjussa näkyvät havainto, syy, korjauscommit ja onnistunut uusintatesti.

Työnäyte Git-repositoryyn ennen rastia: project-docs/testaus.md, kolme virheenkorjausketjua ja korjauscommitit.

Vko 46 · 9.–13.11. – Paketin laatu · Rakenne, jonka toinen ymmärtää

Yhteys kokonaisprojektiin: Viikolla 45 varmistit, että paketti toimii. Nyt siistit rakenteen ja tarkistat LICENSE- ja CREDITS-tiedostot, koska avoimessa julkaisussa joku muu avaa paketin ja jatkaa sitä lisenssin ehdoilla. Ohjaajan tai vertaisen katselmointi näyttää, löytääkö toinen ihminen tiedostot ilman apuasi.

Viikon tavoite: Toinen tekijä löytää paketin tiedostot ilman sinua ja tietää lisenssistä, mitä niillä saa tehdä.

Työvaihe 1 / 3 · Siisti kansiot ja tiedostonimet niin, että paketti toimii kuten ennen.

- □ Etsi kaksi laatuhaittaa. Esimerkiksi epäselvä tiedostonimi, tiedosto väärässä kansiossa tai sounds.json-rivi, jonka äänitiedosto puuttuu.
- □ Kirjaa testi ennen siivousta. Kirjaa testi: paketti latautuu ilman virheitä ja kaikki assetit näkyvät pelissä.
- □ Korjaa yksi haitta kerrallaan. Kopioi korjattu resourcepack-kansio pelin resourcepacks-kansioon ja paina F3 + T. Jos haitta on datapaketissa, kopioi datapack-kansio testimaailmaan ja aja /reload.
- □ Aja testi siivouksen jälkeen. Aja sama testi uudelleen. Tuloksen pitää olla sama kuin ennen.
- □ Kerro siivous commitissa. Tee commit, jonka viesti kertoo, mitä siivosit, esimerkiksi ”Siisti tekstuurien nimet”.

Valmis kun: Kaksi laatuhaittaa on korjattu, ja sama testi menee läpi ennen ja jälkeen.

Tallenna työnäyte: Siivouscommitin linkki ja testin tulokset viikon 46 päiväkirjaan.

Työvaihe 2 / 3 · Tarkista LICENSE- ja CREDITS-tiedostot.

- □ Tarkista LICENSE. Tarkista, että LICENSE on repositoryn juuressa ja vastaa viikolla 35 sovittua.
- □ Käy assetit läpi. Käy läpi jokainen tekstuuri, malli ja ääni. Merkitse listaan, onko se itse tehty vai lisensoitu.
- □ Täydennä CREDITS. Täydennä CREDITS.md niin, että jokainen ulkopuolinen lähde on siinä tekijän ja lisenssin kanssa.
- □ Varmista julkaisuoikeus. Varmista, ettei paketissa ole Mojangin tiedostoja eikä muuta materiaalia, jonka lisenssi ei salli uudelleenjulkaisua.
- □ Vastaa lisenssikysymykseen. Lue oma LICENSE-teksti ja vastaa päiväkirjaan: saako toinen pelaaja julkaista muokatun version paketistasi, ja mitä hänen pitää tehdä?

Valmis kun: LICENSE ja CREDITS.md ovat ajan tasalla, ja vastauksesi lisenssikysymykseen on päiväkirjassa.

Tallenna työnäyte: Commit ja push. Linkit LICENSE- ja CREDITS.md-tiedostoihin viikon 46 päiväkirjaan.

Työvaihe 3 / 3 · Pyydä laatukatselmointi ja selitä ratkaisusi.

- □ Pyydä katselmoija. Pyydä ohjaajaa tai vertaista katselmoimaan paketti eli käymään sen rakenne läpi ja kommentoimaan sitä.
- □ Näytä rakenne. Näytä hänelle, mistä lähdetiedostot, kuten .bbmodel, löytyvät ja mihin uusi tekstuuri lisättäisiin.
- □ Vastaa kommenttiin. Kirjaa kommentin sisältö ja katselmoijan rooli ilman nimeä ja vastaa siihen: korjaa tai perustele, miksi pidät ratkaisun.
- □ Selitä yksi ratkaisu. Selitä suullisesti yksi kohta, jossa käytit tekoälyä apuna. Jos et käyttänyt tekoälyä, selitä yksi JSON-tiedosto omin sanoin.
- □ Tarkista AI-loki. Tarkista, että tekoälyn käyttö on kirjattu sivuston AI-loki-näkymään.

Valmis kun: Päiväkirjassa ovat katselmoijan rooli, hänen kommenttinsa, oma vastauksesi ja se, minkä kohdan selitit.

Tallenna työnäyte: Kommentti, vastaus ja selitetty kohta viikon 46 päiväkirjaan.

Viikon lopputarkistus: Sama testi menee läpi ennen siivousta ja sen jälkeen. LICENSE vastaa sovittua, CREDITS listaa jokaisen ulkopuolisen lähteen, ja katselmointikommenttiin on vastattu.

Työnäyte Git-repositoryyn ennen rastia: siivouscommit, testi ennen ja jälkeen, LICENSE- ja CREDITS.md-tiedostot sekä katselmoinnin kommentti.

Vaihe 5 – Julkaisu ja näyttö

Vko 47 · 16.–20.11. – Julkaisu · Julkaisuehdokas RC1 testissä

Yhteys kokonaisprojektiin: Viikoilla 45 ja 46 paketti testattiin ja siistittiin, joten nyt siihen ei enää lisätä sisältöä. Kokoat ensimmäisen julkaisuehdokkaan (RC1, release candidate 1): version, joka julkaistaan, jos testeissä ei löydy vakavia virheitä. Kahden testaajan havainnot ja niiden luokittelu ratkaisevat, mitä viikolla 48 korjataan ennen julkaisua.

Viikon tavoite: Kaksi testaajaa asentaa julkaisuehdokkaan pelkän asennusohjeen avulla, ja jokaisesta havainnosta on päätös.

Työvaihe 1 / 3 · Jäädytä sisältö ja kokoa ensimmäinen julkaisuehdokas (RC1).

- □ Jäädytä sisältö. Kirjaa päiväkirjaan päivä, josta alkaen pakettiin ei lisätä uutta sisältöä, vaan korjataan vain virheitä. Tätä kutsutaan sisältöjäädytykseksi.
- □ Varmista push. Varmista, että viimeisin commit on pushattu GitHubiin.
- □ Pakkaa kaksi zipiä. Valitse resourcepack-kansion sisältö: pack.mcmeta, pack.png ja assets, ja pakkaa ne zipiksi. Älä pakkaa itse kansiota, muuten peli ei tunnista pakettia. Tee sama datapack-kansiolle.
- □ Nimeä zipit. Esimerkiksi kotikyla-resurssipaketti-rc1.zip ja kotikyla-datapaketti-rc1.zip. RC1 tarkoittaa ensimmäistä julkaisuehdokasta (release candidate 1).
- □ Täydennä asennusohje. Kopioi viikon 41 asennusohje kansioon project-docs/evidence/week-47/ ja lisää datapaketin asennus: zip maailman datapacks-kansioon (Singleplayer → Edit → Open World Folder) ennen maailman avaamista.

Valmis kun: Kaksi zipiä ja asennusohje ovat valmiina, ja ne on koottu samasta commitista.

Tallenna työnäyte: Zipit ja asennusohje kansioon project-docs/evidence/week-47/, commit ja push.

Työvaihe 2 / 3 · Anna kahden testaajan asentaa julkaisuehdokas pelkän ohjeen avulla.

- □ Pyydä kaksi testaajaa. Pyydä ohjaajaa ja yhtä muuta käyttäjää testaamaan julkaisuehdokas (RC1). Muu käyttäjä voi olla toisen ryhmän opiskelija tai toinen ohjaaja.
- □ Anna vain zipit ja ohje. Anna molemmille vain zipit ja asennusohje. Älä neuvo suullisesti.
- □ Pyydä koko sisältö läpi. Pyydä heitä käymään läpi tekstuurit, nimet, mallit, ääni, resepti ja saavutus.
- □ Kirjaa havainnot. Kirjaa jokainen havainto erikseen: testaajan rooli (ohjaaja tai testaaja B, ei nimeä), mitä tapahtui ja missä kohdassa. Kirjaa myös, onnistuiko asennus pelkällä ohjeella.

Valmis kun: Kahden testaajan havainnot on kirjattu erikseen, ja tiedät, onnistuiko asennus ohjeella.

Tallenna työnäyte: Havainnot viikon 47 päiväkirjaan.

Työvaihe 3 / 3 · Päätä jokaisesta havainnosta, korjataanko se ennen julkaisua.

- □ Listaa havainnot. Kirjoita jokainen havainto omalle rivilleen.
- □ Merkitse vakavuus. Vakava (paketti ei toimi tai jokin puuttuu), haitallinen (toimii mutta hankalasti) tai pieni (ulkonäköasia).
- □ Merkitse toistuvuus. Toistuu aina, joskus tai kerran.
- □ Päätä jokaisesta. Korjataan nyt, tunnettu puute (kerrotaan README:ssä) tai myöhemmin (parannus, joka ei haittaa käyttäjää).
- □ Tee korjattavista issuet. Tee jokaisesta korjattavasta havainnosta GitHub-issue.

Valmis kun: Jokaisella havainnolla on vakavuus, toistuvuus ja päätös, ja korjattavista on issuet.

Tallenna työnäyte: Luokiteltu lista ja issueiden linkit viikon 47 päiväkirjaan.

Viikon lopputarkistus: Ensimmäinen julkaisuehdokas (RC1) on koottu yhdestä commitista. Ohjaaja ja toinen käyttäjä ovat asentaneet paketin pelkän ohjeen avulla, ja jokaisella havainnolla on vakavuus, toistuvuus ja päätös.

Työnäyte Git-repositoryyn ennen rastia: RC1-zipit, asennusohje, kahden testaajan havainnot ja luokiteltu korjauslista.

Vko 48 · 23.–27.11. – Julkaisu · v1.0 julki

Yhteys kokonaisprojektiin: Viikon 47 luokiteltu lista kertoo, mitkä virheet korjataan ennen julkaisua. Korjausten jälkeen kirjoitat lataajalle README- ja CHANGELOG-tiedostot ja julkaiset v1.0:n GitHub-releasena eli repositoryn virallisena julkaisuna, jonka versio merkitään tagilla. Vasta ulkopuolisen onnistunut asennus osoittaa, että kuka tahansa saa paketin toimimaan.

Viikon tavoite: Kuka tahansa voi ladata paketin GitHub-releasesta ja asentaa sen ohjeesi avulla ilman sinua.

Työvaihe 1 / 4 · Korjaa vain ne virheet, jotka päätit korjata ennen julkaisua.

- □ Rajaa korjaukset. Avaa viikon 47 luokiteltu lista. Ota työn alle vain havainnot, joiden päätös on ”korjataan nyt”.
- □ Korjaa yksi kerrallaan. Korjaa yksi virhe kerrallaan ja tee jokaisesta oma commit.
- □ Testaa jokainen korjaus. Kopioi korjattu kansio peliin ja aja korjaukseen liittyvä testitapaus sekä regressiotesti eli toinen samaa tiedostoa koskeva testitapaus.
- □ Sulje issuet. Lisää korjattuihin GitHub-issueihin linkki korjauscommitiin ja sulje ne.

Valmis kun: Kaikki ”korjataan nyt” -havainnot on korjattu ja testattu, ja niiden issuet on suljettu.

Tallenna työnäyte: Korjauscommitien linkit viikon 48 päiväkirjaan.

Työvaihe 2 / 4 · Kirjoita lataajalle README- ja CHANGELOG-tiedostot.

- □ Kirjoita asennusohje. Kirjoita README.md-tiedostoon otsikon ”Asennus” alle: resurssipaketti resourcepacks-kansioon, pelin kieleksi suomi ja datapaketti maailman datapacks-kansioon (Singleplayer → Edit → Open World Folder) ennen maailman avaamista.
- □ Lisää perustiedot. Lisää README:hen peliversio, lisenssi ja otsikon ”Tunnetut puutteet” alle viikon 47 tunnetut puutteet.
- □ Lisää kuvat. Lisää README:hen 2–4 kuvakaappausta pelistä.
- □ Kirjoita CHANGELOG. Luo tiedosto CHANGELOG.md ja kirjoita otsikon ”v1.0” alle, mitä paketti sisältää.
- □ Tallenna Gitiin. Tee commit ja push.

Valmis kun: README:ssä ovat asennusohje, peliversio, lisenssi, tunnetut puutteet ja kuvat, ja CHANGELOG.md kertoo, mitä v1.0 sisältää.

Tallenna työnäyte: Linkit README.md- ja CHANGELOG.md-tiedostoihin viikon 48 päiväkirjaan.

Työvaihe 3 / 4 · Julkaise GitHub-release v1.0.

- □ Pakkaa zipit. Valitse resourcepack-kansion sisältö: pack.mcmeta, pack.png ja assets, ja pakkaa ne zipiksi. Älä pakkaa itse kansiota, muuten peli ei tunnista pakettia. Tee sama datapack-kansiolle.
- □ Nimeä zipit versiolla. Esimerkiksi kotikyla-resurssipaketti-v1.0.zip ja kotikyla-datapaketti-v1.0.zip.
- □ Luo release ja tagi. Avaa GitHubissa Releases → Draft a new release. Kirjoita Choose a tag -kenttään v1.0 ja valitse Create new tag. Tagi on nimilappu, joka kiinnittyy yhteen committiin.
- □ Kirjoita julkaisuteksti. Liitä molemmat zipit. Kerro, mitä paketti sisältää, mille peliversiolle se on tehty, miten se asennetaan ja millä lisenssillä se julkaistaan.
- □ Julkaise. Paina Publish release.
- □ Asenna itse releasesta. Lataa julkaistu release ja asenna se puhtaaseen peliin (uusi Installations-asennus omalla Game Directory -kansiolla) pelkän README:n ohjeen avulla. Korjaa ohje, jos jouduit poikkeamaan siitä.
- □ Perusta näyttömatriisi. Luo avun pohjasta project-docs/nayttomatriisi.md: jokaista vaatimusta kohti rivi, jossa ovat tunnus, työnäytteen linkki ja viikko. Täytä julkaisun rivit, esimerkiksi s14 ja v1.0-release, ja tee commit.

Valmis kun: Release v1.0 on julkaistu kahden zipin kanssa, ja olet itse asentanut sen ohjeen avulla puhtaaseen peliin. project-docs/nayttomatriisi.md on perustettu, ja julkaisun rivit on täytetty.

Tallenna työnäyte: Release-linkki viikon 48 päiväkirjaan. project-docs/nayttomatriisi.md commitilla repositoryyn.

Työvaihe 4 / 4 · Pyydä ulkopuolista asentamaan paketti pelkän ohjeen avulla.

- □ Pyydä ulkopuolinen. Pyydä yhtä ihmistä, joka ei ole tekijä, lataamaan v1.0 release-sivulta.
- □ Anna vain README. Pyydä häntä asentamaan paketti pelkän README:n avulla, ilman suullista apua.
- □ Kirjaa epäröinnit. Kirjaa hänen roolinsa, päivä ja jokainen kohta, jossa hän epäröi.
- □ Korjaa ohje. Korjaa epäröintikohdat README:hen, tee commit ja push.
- □ Kuittaa issuet. Kirjoita jokaiseen tämän viikon valmiiseen GitHub-issueen kommentti, jossa on linkki siihen commitiin, jossa työ tehtiin. Sulje issuet vasta sen jälkeen.

Valmis kun: Ulkopuolinen on asentanut paketin pelkän ohjeen avulla, ja hänen epäröintikohtansa on korjattu ohjeeseen.

Tallenna työnäyte: Asentajan rooli, päivä ja havainnot viikon 48 päiväkirjaan.

Viikon lopputarkistus: v1.0-tagin commit vastaa julkaistua versiota. Toinen henkilö on ladannut releasen ja asentanut paketin puhtaaseen peliin pelkän ohjeen avulla. project-docs/nayttomatriisi.md on perustettu.

Työnäyte Git-repositoryyn ennen rastia: release-linkki, v1.0-tagi, README asennusohjeineen, CHANGELOG, ulkopuolisen asennustestin kirjaus ja perustettu project-docs/nayttomatriisi.md.

Vko 49 · 30.11.–4.12. – Näyttö · Aineisto ja luovutus

Yhteys kokonaisprojektiin: Paketti on julkaistu, eikä siihen lisätä enää sisältöä. Kokoat viikkojen päiväkirjasta, testeistä ja releasesta näyttöaineiston, jossa jokaisella vaatimuksella on tarkka linkki työnäytteeseen. Demo ja luovutus päättävät projektin.

Päivämäärät:

- **Demo ja luovutus:** viimeistään pe 4.12.2026

Viikon tavoite: Arvioija löytää jokaisen osaamisvaatimuksen työnäytteen yhdellä tarkalla linkillä.

Avaa näyttömatriisi

Avaa ja lataa AI-loki

Työvaihe 1 / 5 · Tarkista projektipäiväkirja.

- ☐ Tarkista kirjatut viikot. Avaa Projektipäiväkirja-näkymä ja tarkista, että jokainen viikko on merkitty kirjatuksi.
- ☐ Täydennä puuttuvat. Täydennä puuttuvat kentät viikkonäkymissä.
- ☐ Kokeile linkit. Tarkista jokaisen viikon Missä työnäyte on? -kentästä, että linkki aukeaa.

Valmis kun: Kaikki 15 viikkoa on kirjattu, ja jokaisen viikon työnäytelinkki aukeaa.

Tallenna työnäyte: Täydennetyt viikkomerkinnät sivuston projektipäiväkirjassa. Lataat tiedoston repositoryyn työvaiheessa 5.

Työvaihe 2 / 5 · Kirjoita itsearviointi omiin työnäytteisiin nojaten.

- ☐ Valitse kolme vahvuutta. Valitse projektista kolme asiaa, jotka osaat nyt hyvin.
- ☐ Liitä työnäytteet. Liitä jokaiseen vahvuuteen työnäyte: linkki commitiin, testitapaukseen tai päiväkirjan viikkoon.
- ☐ Kirjoita kehitysaskel. Kirjoita yksi asia, jota haluat kehittää seuraavaksi, ja miten aiot kehittää sitä.
- ☐ Kirjoita itse. Kirjoita itsearviointi itse. Älä käytä tekoälyä tekstin kirjoittamiseen.

Valmis kun: Viikon 49 päiväkirjassa on kolme vahvuutta työnäytteineen ja yksi kehitysaskel.

Tallenna työnäyte: Itsearviointi viikon 49 päiväkirjaan.

Työvaihe 3 / 5 · Liitä jokaiseen näyttömatriisin kohtaan tarkka linkki.

- ☐ Katso puuttuvat kohdat. Avaa project-docs/nayttomatriisi.md ja sivun Näyttömatriisi-näkymä eli oma muistilistasi osaamisvaatimuksista. Katso, miltä vaatimuksilta rivi tai rasti vielä puuttuu.
- ☐ Etsi työnäytteet. Etsi jokaiselle puuttuvalle kohdalle työnäyte: issue, tiedosto, commit, tagi, kuva tai testitapaus.
- ☐ Kirjaa tarkat linkit. Kirjoita nayttomatriisi.md-tiedostoon jokaiselle vaatimukselle rivi: tunnus, tarkka linkki ja viikko. Linkin pitää avata juuri se kohta, ei repositoryn etusivua.
- ☐ Rastita vaatimus. Rastita vaatimus sivun Näyttömatriisi-näkymässä, kun sen rivi on tiedostossa.

Valmis kun: project-docs/nayttomatriisi.md:ssä on rivi jokaiselle 32 vaatimukselle, ja jokainen linkki aukeaa oikeaan kohtaan.

Tallenna työnäyte: project-docs/nayttomatriisi.md commitilla repositoryyn. Linkki tiedostoon viikon 49 päiväkirjaan.

Työvaihe 4 / 5 · Harjoittele 8–10 minuutin demo.

- ☐ Kirjoita demon runko. Kirjoita runko kuutena kohtana: paketti pelissä, yksi tekstuuri- tai mallityö, resepti ja saavutus, yksi virheenkorjaus, Git-historia ja AI-loki.
- ☐ Harjoittele kuulijalle. Esitä demo kerran harjoituksena toiselle ihmiselle ja ota aika. Tavoite on 8–10 minuuttia. Kirjaa kuulijan rooli ja yksi asia hänen palautteestaan.
- ☐ Harjoittele selitys. Harjoittele, miten selität yhden JSON- tai mcfunction-tiedoston omin sanoin.
- ☐ Käytä versiota v1.0. Varmista, että demossa käytät julkaistua versiota v1.0.

Valmis kun: Demo kestää 8–10 minuuttia ja käy läpi kaikki kuusi kohtaa.

Tallenna työnäyte: Demon runko viikon 49 päiväkirjaan.

Työvaihe 5 / 5 · Luovuta paketti ja aineisto viimeistään pe 4.12.2026.

- ☐ Vie päiväkirja repositoryyn. Paina Projektipäiväkirja-näkymän painiketta Lataa koko päiväkirja (.md). Korvaa ladatulla tiedostolla repositoryn project-docs/projektipaivakirja.md. Tee commit ja push.
- ☐ Tarkista AI-loki. Avaa ladattu tiedosto ja tarkista, että sen lopussa on otsikko AI-loki ja omat merkintäsi.
- ☐ Pyydä tarkistus. Pyydä toista ihmistä avaamaan release ja repository. Kirjaa, löysikö hän kaiken.
- ☐ Luovuta. Luovuta paketti, repository, projektipäiväkirja ja näyttöaineisto ohjaajalle viimeistään pe 4.12.2026.

Valmis kun: Aineisto on luovutettu viimeistään pe 4.12.2026, ja toinen ihminen on tarkistanut sen.

Tallenna työnäyte: Luovutuksen päivä viikon 49 päiväkirjaan.

Viikon lopputarkistus: project-docs/nayttomatriisi.md:ssä jokaisella arviointikohdalla on avautuva tarkka linkki. Projektipäiväkirja ja AI-loki ovat repositoryssa, ja demo käyttää samaa jäädytettyä v1.0-versiota.

Työnäyte Git-repositoryyn ennen rastia: julkaistu paketti, repository, project-docs/projektipaivakirja.md, project-docs/nayttomatriisi.md, AI-loki ja demo.

Viimeiset viisi päivää

Ma 30.11. · Sisältöjäädytys — viimeinen hyväksytty versio

Ti 1.12. · Aineisto — päiväkirja, testit ja linkit

Ke 2.12. · Harjoittelu — 8–10 min demo ja itsearviointi

To 3.12. · Puskuri — tarkistus toisen henkilön kanssa

Pe 4.12. · LUOVUTUS — paketti, repository, projektipäiväkirja ja näyttö

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

asset	*Paketin osa.* Pelissä näkyvä tai kuuluva paketin osa: tekstuuri, 3D-malli, ääni tai skriptattu lisä. Tämä projekti etenee yksi asset kerrallaan.
asset pack	*Assettien kokoelma.* Assettien kokoelma, joka asennetaan peliin yhtenä pakettina. Tässä projektissa asset pack tarkoittaa resurssipakettia ja datapakettia yhdessä.
resurssipaketti	*Resource pack.* Paketin osa, joka muuttaa sitä, miltä peli näyttää ja kuulostaa: tekstuurit, mallit, äänet ja nimet. Kansion nimi levyllä on resourcepack.
datapaketti	*Data pack.* Paketin osa, joka lisää peliin sääntöjä: reseptit, saavutukset ja funktiot. Kansion nimi levyllä on datapack.
pack.mcmeta	*Paketin käyntikortti.* Pieni JSON-tiedosto paketin juuressa. Se kertoo pelille paketin kuvauksen ja pack_format-arvon. Ilman sitä peli ei tunnista pakettia.
pack_format	*Paketin versionumero.* Numero pack.mcmeta-tiedostossa. Se kertoo pelille, mille Minecraft-versiolle paketti on tehty. Resurssipaketilla ja datapaketilla on eri arvo samalle peliversiolle.
mcfunction	*Komentotiedosto.* Tekstitiedosto, jossa on pelin omia komentoja rivi riviltä. Peli ajaa rivit järjestyksessä /function-komennolla. Tämän projektin skriptaus tehdään näin, ei ohjelmointikielellä.
JSON	*Tekstimuotoinen määrittely.* Yksinkertainen tekstimuoto, jolla paketin määrittelyt kirjoitetaan: kielitiedosto, mallit, reseptit ja saavutukset. Pilkku tai lainausmerkki väärässä paikassa rikkoo koko tiedoston.
nimiavaruus vko 40	*Paketin oma lokero.* Kansio, jonka nimi on paketin oma nimi. Sen sisällä ovat sinun tiedostosi. Se erottaa oman sisällön pelin omasta sisällöstä, esimerkiksi teema:kyla.kello.
repository	*Projektin varasto GitHubissa.* Projektin koko tiedosto- ja versiohistoria GitHubissa. Tämän projektin repository on julkinen ensimmäisestä commitista asti.
commit	*Tallennettu muutos.* Yksi Gitiin tallennettu muutos, jolla on oma viesti ja tunniste. Commit-linkki on tämän projektin tavallisin työnäyte.
push vko 34	*Lähetys GitHubiin.* Omalla koneella tehtyjen committien lähettäminen GitHubiin. Ennen ensimmäistä pushia tarkistetaan, ettei mukana ole henkilötietoja – Git-historia on pysyvä.
GitHub-issue	*Yksi asset tai muutos repositoryssa.* Repositoryn Issues-listaan kirjattu yksi rajattu työ, esimerkiksi ”Piirrä kolme blokkitekstuuria”: otsikko, arvio, prioriteetti ja Valmis kun -ehto. Yksi issue on noin puolen tai yhden päivän työ. Issue ei ole sama kuin sivun työvaihe: yksi työvaihe voi koskea useaa issueta.
asset-työkierto	*Yhden assetin työtapa.* Työtapa-sivun kuusi askelta, joilla teet yhden assetin tai issuen valmiiksi: valitse, suunnittele, toteuta, testaa pelissä, tallenna commitiksi ja kirjaa. Viikon työvaiheet kertovat, mitä teet; kierto kertoo, miten teet jokaisen assetin.
haara vko 43	*Branch.* Rinnakkainen kopio projektista, jossa yksi muutos tehdään rauhassa valmiiksi. Main-haara pysyy koko ajan toimivana.
merge vko 43	*Haarojen yhdistäminen.* Haaran muutosten yhdistäminen takaisin main-haaraan. Merge tehdään vasta, kun muutos on testattu.
pull request vko 43	*Pyyntö yhdistää haara.* GitHubin tapa tehdä merge: pyyntö yhdistää haara main-haaraan. Pyynnössä muutokset voi lukea läpi ja kommentoida ennen hyväksymistä.
tagi vko 48	*Version nimilappu.* Gitin nimilappu, joka kiinnitetään yhteen committiin, esimerkiksi v1.0. Tagi kertoo, mikä versio julkaistiin.
GitHub-release	*Julkaistu versio.* GitHubissa julkaistu versio, jonka kuka tahansa voi ladata. Tässä projektissa releaseen liitetään zip-paketit, asennusohje ja kuvat.
P0 vko 35	*Pakollinen ydin.* Pakollinen (P0) on sisältö, jonka on valmistuttava ennen mitään lisäsisältöä. Tämän projektin pakollinen sisältö on 5 tekstuuria (3 blokkia ja 2 esinettä), omat nimet, 1 Blockbench-malli, 1 isompi malli tai mobin uusi ilme, 1 resepti, 1 saavutus ja 1 palkintofunktio.

P1	*Tärkeä jatkosisältö.* Sisältö, joka tehdään vasta, kun P0 toimii. P1 on tärkeää mutta ei estä julkaisua.
P2	*Valinnainen lisä.* Sisältö, joka voidaan jättää kokonaan pois, jos aika loppuu. P2 ei vaikuta näytön läpimenoon.
T01 vko 45	*Testitapauksen tunnus.* T tarkoittaa testitapausta ja numero sen järjestystä: T01 on ensimmäinen testitapaus, T02 toinen. Tunnukset T01–T12 annetaan viikon 45 testaustaulukossa. Viikkojen omat testit ovat vain testi 1, testi 2 ja niin edelleen.
UV-kartta vko 38	*Tekstuurin ja mallin pintojen vastaavuus.* UV-kartta kertoo, mikä kohta tekstuurikuvasta piirtyy mihinkin mallin pintaan. Blockbench tekee sen puolestasi, kun maalaat Paint-tilassa.
mobi vko 39	*Pelin hahmo tai eläin.* Mobi on Minecraftin liikkuva olento, esimerkiksi creeper, lehmä tai kyläläinen. Mobin ilmeen voi muuttaa piirtämällä sille uuden tekstuurin.
vertainen vko 39	*Toinen opiskelija.* Vertainen on toinen opiskelija, joka keskustelee kanssasi, testaa pakettiasi tai katselmoi työtäsi.
CREDITS vko 40	*Lähdeluettelo.* CREDITS.md on repositoryn juuressa oleva luettelo kaikesta, mitä pakettiin on tuotu muualta: tekijä, lähde ja lisenssi. Jos kaikki on itse tehtyä, CREDITS kertoo senkin.
katselmointi vko 41	*Työn yhteinen tarkastus.* Katselmoinnissa toinen ihminen kokeilee pakettia tai käy sen rakenteen läpi ja antaa palautetta. Viikolla 41 testaajat katselmoivat väliversion, viikolla 46 ohjaaja tai vertainen katselmoi rakenteen.
advancement vko 44	*Saavutus.* Advancement eli saavutus on datapaketin JSON-tiedosto. Se näyttää pelaajalle ilmoituksen, kun hän tekee jotain tiettyä, ja voi ajaa palkintofunktion.
regressiotesti vko 45	*Vanhan toiminnan uusintatesti.* Regressiotesti on toinen testitapaus, joka koskee samaa tiedostoa kuin korjaus. Kun se menee läpi, tiedät, ettei korjaus rikkonut muuta.
RC vko 47	*Release candidate, julkaisuehdokas.* Lähes valmis versio, jota testataan sellaisena kuin se aiotaan julkaista. RC1 on ensimmäinen julkaisuehdokas. Siihen ei enää lisätä uutta sisältöä.
backlog vko 35	*Priorisoitu issue-lista.* Repositoryn kaikki tekemättömät GitHub-issuet tärkeysjärjestyksessä. Jokaisella issuella on prioriteetti (P0, P1 tai P2), työmääräarvio ja Valmis kun -ehto.
moodboard vko 35	*Ilmeen kuvakollaasi.* Kooste väripaletista ja referenssikuvista, joka lukitsee paketin ilmeen ennen pikselityötä. Moodboardiin palataan aina, kun mietit, sopiiko uusi asset teemaan.
CC	*Creative Commons.* Lisenssiperhe kuville, äänille ja muulle sisällölle. CC BY vaatii tekijän mainitsemisen. CC BY-SA vaatii lisäksi, että muokatut versiot julkaistaan samalla lisenssillä. CC0 luovuttaa työn vapaaseen käyttöön.
MIT	*MIT-lisenssi.* Yleinen ja salliva ohjelmistolisenssi. Sopii datapaketin skripteille: muut saavat käyttää ja muokata koodia, kunhan lisenssiteksti pysyy mukana.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

□ **Kehitysympäristö** – Blockbench- ja editorikuva, peliin ladattu paketti ja repository.
□ **Virheenkorjaus** – Kolme täydellistä ketjua aidoista havainnoista tai opettajan merkitsemistä vikatehtävistä.
□ **Toimintojen testaus** – 12 testiä odotettuine ja toteutuneine tuloksineen.
□ **Rakenteinen ohjelmointi** – Datapaketin funktiot ja reseptit jaettu selkeisiin vastuisiin omassa nimiavaruudessa.
□ **Ylläpidettävä koodi** – Selkeät tiedostonimet, siisti JSON ja siivouscommit.
□ **Käyttöliittymä** – Pelinäkymän luettavuus: tekstuurien ja nimien ennen/jälkeen-kuvat.
□ **Toimintojen toteutus** – Asset-tehtävät, valmis kun -ehdot, issuet ja commitit.
□ **Tehtävistä sopiminen** – Tehtävälista ja kokousmuistio.
□ **Ratkaisuvaihtoehdot** – Kahden toteutustavan vertailu (vko 39) ja päätös.
□ **Ratkaisujen arviointi** – Paketti- tai versiokatselmoinnin loki.
□ **Oma toiminta tiimissä** – Itsearviointi ja seuraava kehitysaskel.

Ohjelmistokehittäjänä toimiminen

□ **Asiakkaan tarpeet** – Kysymyslista ohjaajalle, aloitusmuistio ja kuvaus paketin kohdeyleisöstä.
□ **Asiakaslähtöinen viestintä** – Ymmärrettävä yhteenveto vaihtoehdoista ja rajoista sekä lataajalle kirjoitettu README ja asennusohje.
□ **Version katselmointi** – Lokit viikoilta 41 ja 47.
□ **Priorisointi** – P0/P1/P2-backlog ennen ja jälkeen palautteen.
□ **Tehtäviksi jakaminen** – 0,5–1 päivän issuet ja nimetyt tekijät.
□ **Suunnittelu ja arviointi** – Työmääräarvio verrattuna toteumaan.
□ **Toimintalogiikka** – Reseptit, funktiot, saavutus ja niiden ehdot.
□ **Tietovaraston valinta** – JSON-tiedostojen rakenne ja perustelu.
□ **Yhteys tietovarastoon** – Tekstuurien, mallien ja datan kytkeytyminen pelin resursseihin nimiavaruuksien kautta.
□ **Rajapinnat ja tieto** – Pakettirajapinta: pack.mcmeta, tiedostopolut ja niitä vastaavat tiedostot.
□ **Tietoturva** – Oma lisenssi ja kolmansien osapuolten lisenssit, luotettavat lataukset ja se, ettei julkisessa repositoryssa ole henkilötietoja tai salaisuuksia.
□ **Versionhallinta** – Jatkuva Git-historia ja toimiva main.
□ **Osan liittäminen** – Testattu merge tai pull request.
□ **Tuotantojulkaisu** – Toimiva v1.0-release-linkki.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

□ **Kehittämisympäristön käyttöönotto** – Blockbench, VS Code, sovittu Minecraft-versio ja paketin lataus peliin.
□ **Kirjaston mahdollisuudet ja rajoitteet** – Viikon 45 kirjaus pakettijärjestelmän rajoituksista ja viikon 48 julkaisupäätökset tunnettuine puutteineen.
□ **Kirjaston toiminnot ja työkalut** – Blockbenchin mallinnus, UV-editointi ja esikatselu committeina ja kuvina.
□ **Ulkoiset komponentit** – Paletit, äänet ja referenssit tuotuina; lähteet ja lisenssit kirjattu CREDITS-tiedostoon.
□ **Suunnittelu, toteutus ja testaus kirjastolla** – Asset-pack-suunnitelman päätökset ja backlog, Assetit 1–7 committeina, testiloki T01–T12.
□ **Julkaisu asiakkaan ympäristöön** – Julkinen GitHub-release, jonka toinen henkilö on asentanut peliinsä pelkän ohjeen avulla.
□ **Ohjelmiston dokumentointi** – Lataajalle: README, asennusohje, LICENSE, CREDITS ja CHANGELOG. Arviointiin: asset-pack-suunnitelma.md ja projektipäiväkirja.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.